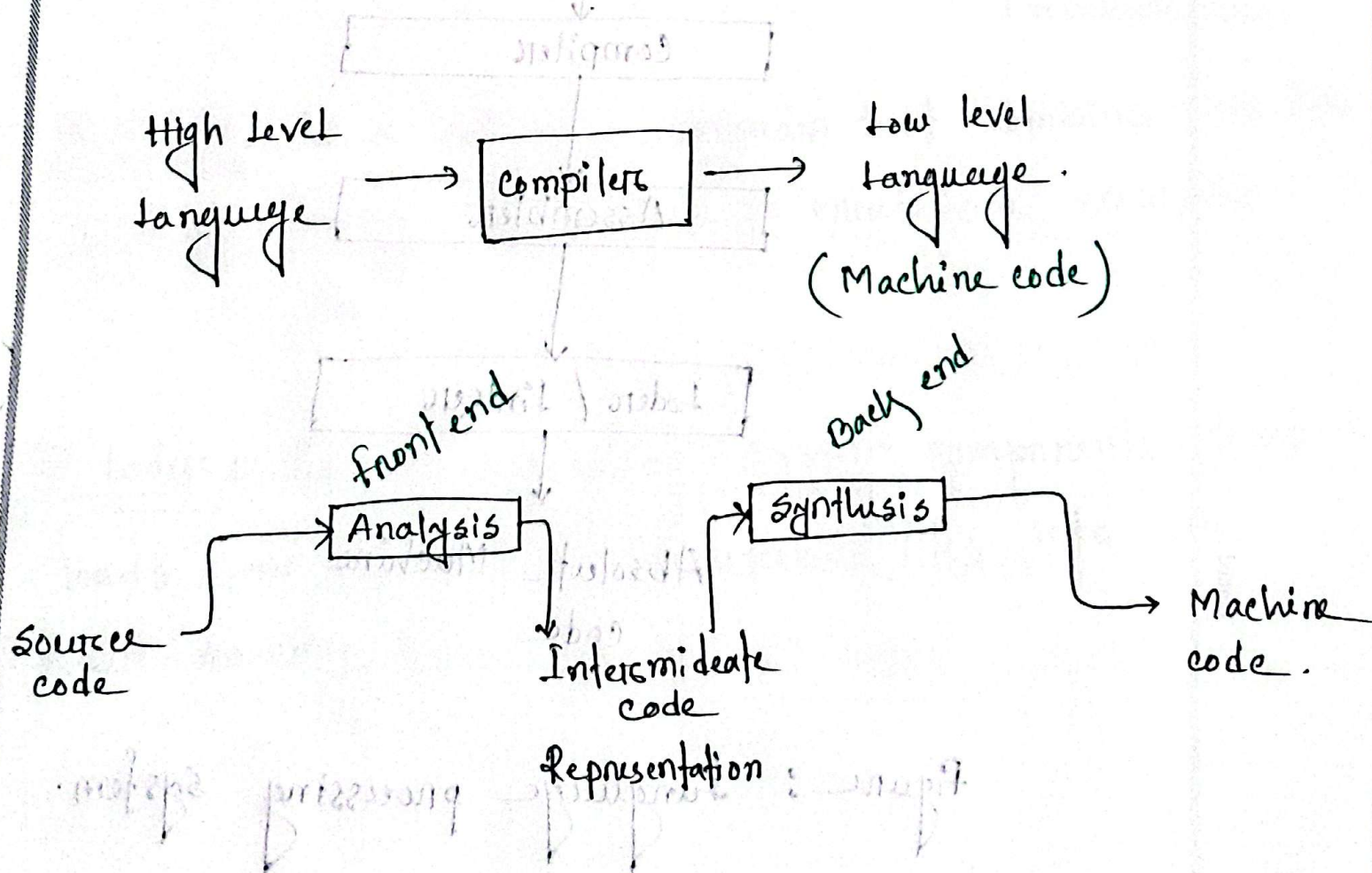


Compiler Design — (Crash course).

- Lexical Analysis
- Parser (Syntax Analyzer) *** (most important)
- Semantic Analysis
- Intermediate code generator ** (important)
- Code optimization

Introduction to compiler design → (Lecture-01)

The compiler is a software that converts a program written in high level language to a low level language.



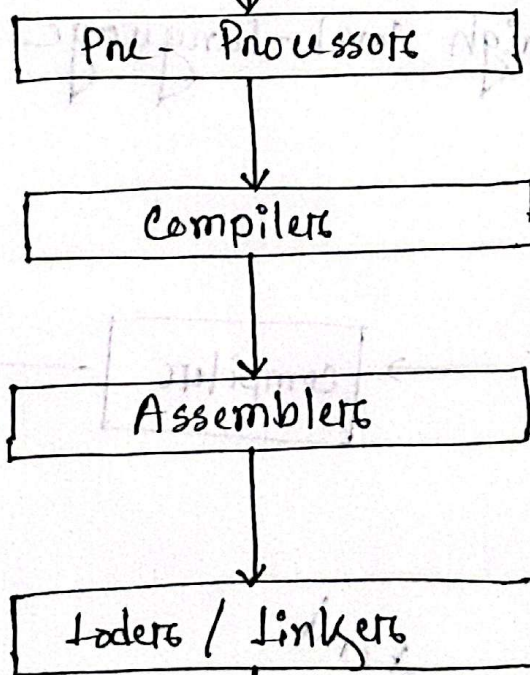
Analysis Divided into

- Lexical Analyzer.
- Syntax Analyzer.
- Semantic Analyzer.
- Intermediate code generators.

Synthesis Divided into

- Code optimizer.
- Target code generation.

High level language

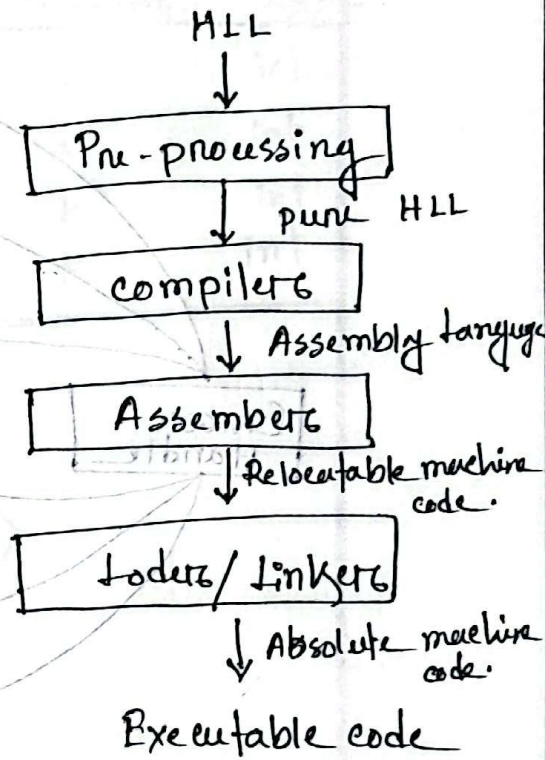


Absolute Machine code

Figure: Language processing system.

Language processing system → (Lecture-02)

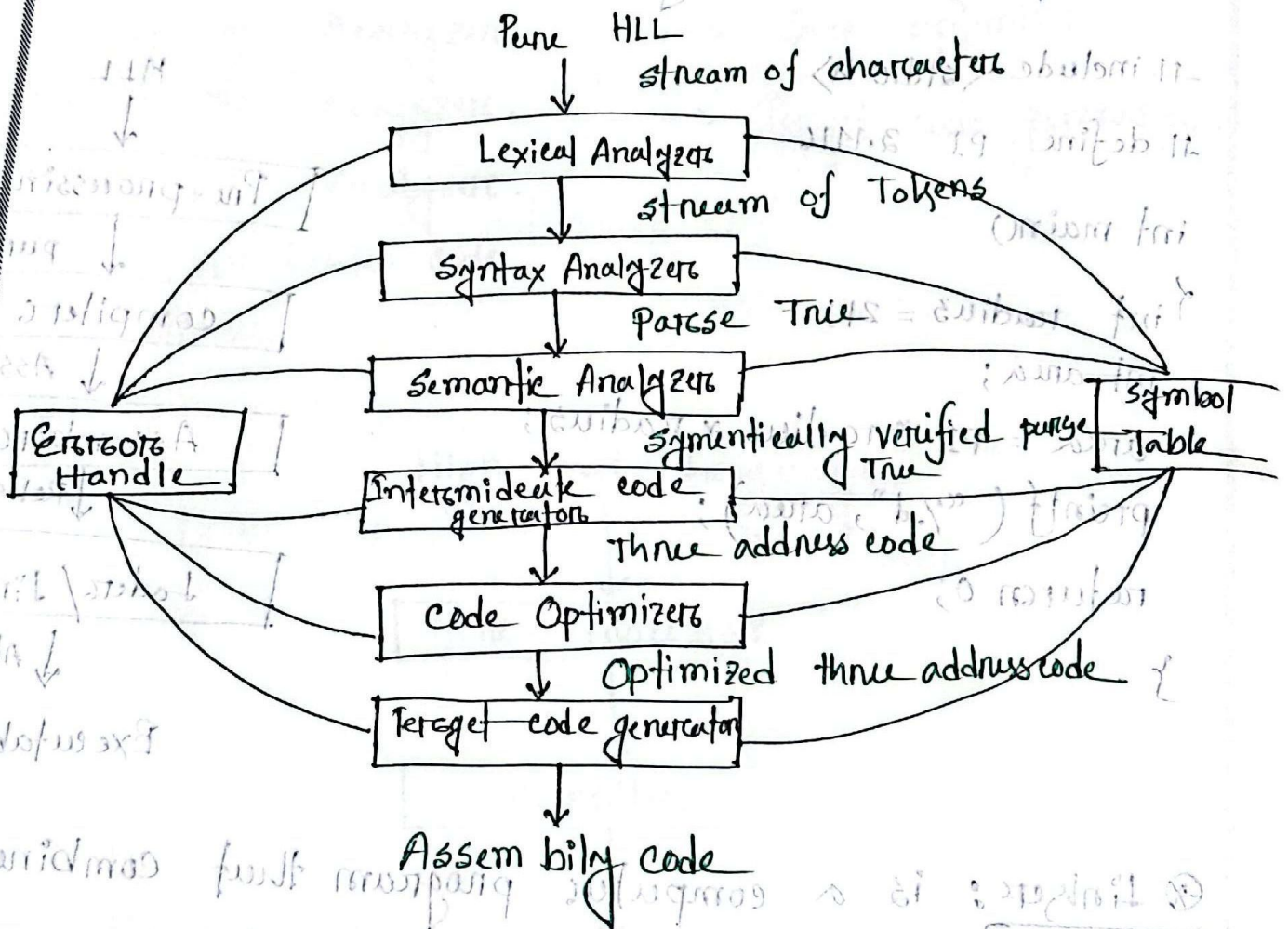
```
#include <stdio.h>
#define PI 3.1416
int main()
{
    int radius = 2;
    int area;
    area = PI * radius * radius;
    printf("%d", area);
    return 0;
}
```



⊗ Linker: is a computer program that combines and links many objects files to create one executable file.

⊗ Loader: Is an operating system components that loads and executes executables files into the main memory.

▣ Phases of compiler → (Lecture -03)



Example: Variable বা user define গুলো ফাংশন-ভিত্তিক-

সেটুলোকে identify হিসাবে লিখতে হবে।

Lexical analyzer takes pure high level language
as stream of characters input and generates
output as stream of Tokens.

(i) High level language (pure HLL) (iii)

int x, a, b, c;

x = a + b * c;

Symbol table

S.No	Variable	Type
1.	x	int
2.	a	int
3.	b	int
4.	c	int

(ii) Lexical Analyzer

$\langle id, 1 \rangle = \langle id, 2 \rangle + \langle id, 3 \rangle * \langle id, 4 \rangle$

Here

$\langle id, 1 \rangle = "x"$: Identifier (1)

"=" : Assignment operators

$\langle id, 2 \rangle = "a"$: Identifier (2)

"+" : Addition Operators

$\langle id, 3 \rangle = "b"$: Identifier (3)

"*" : Multiplication operators

$\langle id, 4 \rangle = "c"$: Identifier (4)

(iii) Syntax Analyzer →

id	val	op	val	op	val
1	5	+	2	*	3
2	5	+	2	*	3
3	5	+	2	*	3
4	5	+	2	*	3

$$\langle \text{t.bi} \rangle * \langle \text{e.bi} \rangle + \langle \text{s.bi} \rangle = \langle \text{t.bi} \rangle$$

① initial : "x" $\langle \text{t.bi} \rangle$

② initial : "x"

③ initial : "x" $\langle \text{s.bi} \rangle$

④ initial : "x"

(iv) Symantic Analyzer →

Symantically verified parse tree. Input string

parse tree to check if it is a valid variable or data

type or not. And check if it is a valid expression

condition check and finally generate verified parse tree.

(v) Intermediate code generator →

input → Symantically verified parse tree

output → Three address code (প্রধানত ৩ ধরনের ভেরিয়েবল ব্যবহার করতে পারবে)

$$t_1 = b * c$$

$$t_2 = a + t_1$$

$$x = t_2$$

Every line ৩ ধরনের ভেরিয়েবল ব্যবহার হয়। প্রত্যেক Three address code বলা হয়।

(vi) code Optimization →

input → Three address code.

Output → Optimized three address code

$$t_1 = b * c$$

$$x = a + t_1$$

(vii) Target code Generation →

input → Optimized three address code

output → Assembly code generate

~~MUL~~ AL

MUL AL, BL

ADD CL, AL

MOV DL, CL

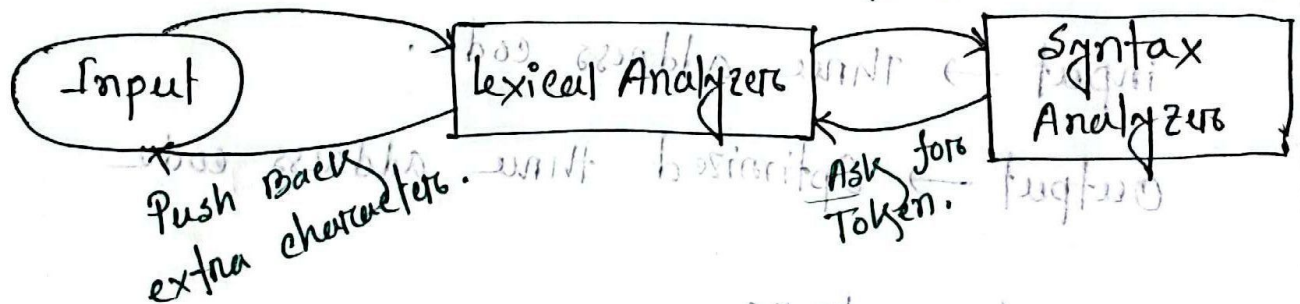
Lexical Analyzer → (Lecture-09)

Lexical analyzer is the first phase of compiler, also known as scanner. It converts pure HLL into a sequence of Tokens.

→ Lexical automata Analyzer can be implemented with the Deterministic finite automata. (DFA)

Read characters.

Tokens.



Token: is a sequence of characters that can be treated as a unit in the grammar of the programming language.

Tokens may be

- Keyword. (pre defined like: int, if, else, while etc)
- Identifier. (users define, variable)
- Operators. (Assignment operators, Arithmetic operators)
- Constants. (Fixed value)
- Punctuators. (', ', ')

Example

```
int a = 10;
```

Here,

int : Keyword.

a : Identifier.

= : Assignment operators 'x', 'x'

10 : Constant

; : Punctuators.

All are the tokens.

lexical analyzer

1. Input preprocessing (remove white space, comment)
2. Tokenization (Input text to sequence of tokens)
convert

3. Token classification. (সহজ keyword, identifier, punctua
(যেমন: int, float, char, etc. for - এবং identify করে।)

4. Token validation. (Lexers check করে যে tokens
generate হচ্ছে এবং programming
language এর Rule অনুযায়ী
হচ্ছে কিনা)।

5. Error Detection.

Count number of Tokens → (Lecture-05)

"x", 'x' → Double quote এবং single quote এর মধ্যে

একটি বা একাধিক token হিসাবে বিবেচনা করা হয়।

→ pointer কেবল আলাদা টোকেন ধরা হয়।

code এর ভিতর যদি কোনো syntax error থাকে তবে একটি
token convert হয়ে যায় null, কোনো প্রকার পড়বে না।

Confusing code → Comment token এর মধ্যে আলাদা।

(যদি comment)

- int main() → 4 token
 → int main() → 4 token
 → int a == 20; → 5 token
 → int b <= 10; → 5 token
 → int a = 5; → 6 token
 → char a = 'f'; → 6 token
 → char *a = "xyz"; → 6 token (pointer to string token)
 → int /* KALLOI */ d = 100; → 5 token (comment count not 2/3)
 → A == B; → 5 token

N.B

printf("xyz

↑
This situation is known as token error because

("") works on a pairs but in here one single quote

are integrated.

int main()

United

প্রধানতঃ প্রথম মাঝে Count শুরু হয় ০ ১ ২ ৩ ৪, প্রথম

এমে প্রধানতঃ space পাড়ে, মাঝে এটি একটি keyword হিসাবে গণ্য
করা হয়। এখানে একটি DFA (Deterministic finite Automata)

গণ্য হয়।

Q what is Grammar (lecture-06) →

In compiler design, grammar refers to the formal rules that define the syntax of a programming language. It defines the structures of a valid sentences or statement, in the language.

⇒ Grammar consists of a set of production rules that describe how valid language constructs can be formed.

A grammar has 4 tuples

$$G = (V, T, P, S)$$

where,

capital letter) V = Variables/ non-terminals. // V = Variables/ non-terminals.
 small letter) T = Terminals. // string to terminate V by T .
 P = Productions. // P = Rule define V by T .
 S = start symbols. // S = start symbols.

Suppose an example of grammar $\rightarrow$

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

Variables, $V = \{S, A, B\}$

Terminals, $T = \{a, b\}$

Productions, $P = \left\{ \begin{array}{l} S \rightarrow AB \\ A \rightarrow a \\ B \rightarrow b \end{array} \right\}$

$$L = ab.$$

Start, $S = \{S\}$
Symbols

Q what is language (lecture - 07) $\rightarrow$

In compiler design the language of a grammar refers to the set of all possible strings that can be generated by the grammar.

Grammar G is defined by

$$G = (V, T, P, S)$$

Language $L(G)$:

$$\{ \alpha \mid \alpha \in L(G) \}$$

The language of a grammar G is a set of all strings composed the terminal symbols that can be derived from the start symbol S by applying the production rules in P .

There are two types of language.

- (i) Infinite language.
- (ii) finite language.

* finite language $\rightarrow$ (Example)

$\rightarrow$ language $\rightarrow$ (a finite limit) $\rightarrow$!

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$V = \{S, A, B\}$$

$$T = \{a, b\}$$

$$P = \left\{ \begin{array}{l} S \rightarrow AB \\ A \rightarrow a \\ B \rightarrow b \end{array} \right\}$$

$$L(G) = \{ab\}$$

$$S = \{S\}$$

* Infinite $\rightarrow$ (Example)

এই language এর continuation মাঝে মাঝে stop করা সম্ভব নয়। Infinite language হল।

$$b \rightarrow a^s / \epsilon$$

$$= \{ \epsilon, a, aa, aaa, \dots \}$$

$$= \{ \epsilon, a, a^2, a^3, \dots, a^n \}$$

Leftmost Derivation (Lecture - 8) $\rightarrow$

$\rightarrow$ In compiler design, a leftmost derivation is a way of deriving a string from a context free grammar by always expanding the left-most non-terminal first.

$\rightarrow$ In a leftmost derivation, at each step, the left most non-terminal into the current string is replaced one of the production rules.

$$[a^i * b^i + b^i \leftarrow a]$$

context free grammars and formal system
 defining programming language as
 syntax define karo.

Example:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

All are }
 Context free
 grammars.

Deriving the string "id + id * id". Using leftmost
 derivation.

solution $\rightarrow$

$$E \rightarrow E + T$$

$$E \rightarrow T + T$$

$$E \rightarrow F + T$$

$$E \rightarrow \text{id} + T$$

$$E \rightarrow \text{id} + T * F$$

$$E \rightarrow \text{id} + F * F$$

$$E \rightarrow \text{id} + \text{id} * F$$

$$E \rightarrow \text{id} + \text{id} * \text{id}$$

Example: 2

$E \rightarrow E + E \mid E * E \mid id$ // context free grammar.

Deriving the string "id + id * id", using leftmost derivations.

solution

$E \rightarrow E + E$

$E \rightarrow id + E$

$E \rightarrow id + E * E$

$E \rightarrow id + id * E$

$E \rightarrow id + id * id$

Rightmost Derivations (Lecture-09) →

উদাহরণে যেখানে আমরা context free grammar থেকে string generate করতে যাব তখন priority হবে প্রকরণ টান থেকে। Leftmost derivations এর প্রকরণ বিপরীত Rule follow করে।

Example : 1

Given that,

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

Deriving the string "id + id * id", using rightmost derivation.

Solution $\rightarrow$

$$E \rightarrow E + T$$

$$E \rightarrow E + T * F$$

$$E \rightarrow E + T * \text{id}$$

$$E \rightarrow E + F * \text{id}$$

$$E \rightarrow E + \text{id} * \text{id}$$

$$E \rightarrow T + \text{id} * \text{id}$$

$$E \rightarrow F + \text{id} * \text{id}$$

$$\boxed{E \rightarrow \text{id} + \text{id} * \text{id}}$$

Example : 2

$$E \rightarrow E + E \mid E * E \mid id$$

Deriving the string "id + id * id" using rightmost derivators.

solution $\rightarrow$

$$E \rightarrow E + E$$

$$E \rightarrow E + E * E$$

$$E \rightarrow E + E * id$$

$$E \rightarrow E + id * id$$

$$E \rightarrow id + id * id$$

Example of leftmost & rightmost derivation

(lecture - 10) $\rightarrow$

Given that, $E \rightarrow E + T \mid E - T \mid T$.

$T \rightarrow T * F \mid T / F \mid F$.

$F \rightarrow (E) \mid id$.

Deriving the string "id * (id + id)" using leftmost and rightmost derivation.

By using leftmost derivations $\rightarrow$

$$E \rightarrow T$$

$$E \rightarrow T * F$$

$$E \rightarrow F * F$$

$$E \rightarrow id * F$$

$$E \rightarrow id * (E)$$

$$E \rightarrow id * (E + T)$$

$$E \rightarrow id * (T + T)$$

$$E \rightarrow id * (F + T)$$

$$E \rightarrow id * (id + T)$$

$$E \rightarrow id * (id + F)$$

$$E \rightarrow id * (id + id)$$

By using Right-most derivation:

$$E \rightarrow T$$

$$E \rightarrow T * F$$

$$E \rightarrow T * (E)$$

$$E \rightarrow T * (E + T)$$

$$E \rightarrow T * (E + F)$$

$$E \rightarrow T * (E + id)$$

$$E \rightarrow T * (T + id)$$

$$E \rightarrow T * (f + id)$$

$$E \rightarrow T * (id + id)$$

$$E \rightarrow f * (id + id)$$

$$E \rightarrow id * (id + id)$$

Parse Tree (Lecture -11) →

Graphical representation of derivation is called parse tree. (Exam & assignment string derivation → then parse tree ତିଆରି କରା ଉଠେ).

Given that,

c.f grammar: $E \rightarrow E + E \mid E * E \mid id$

string: $id + id * id$

By using leftmost derivation →

$$E \rightarrow E + E$$

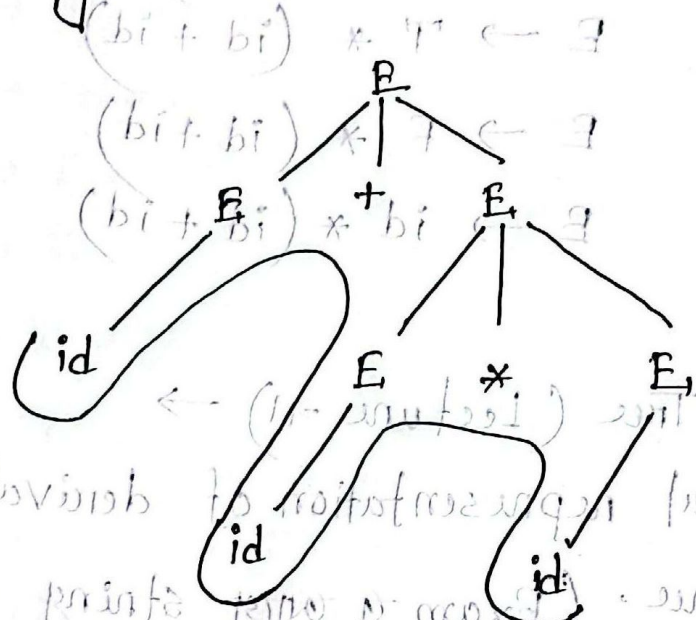
$$E \rightarrow id + E$$

$$E \rightarrow id + E * E$$

$$E \rightarrow id + id * E$$

$$E \rightarrow id + id * id.$$

Now we configure or generate a parse tree according to the derivations. (Leftmost Derivation)



Another way (By using Rightmost Derivations-

$$E \rightarrow E + E$$

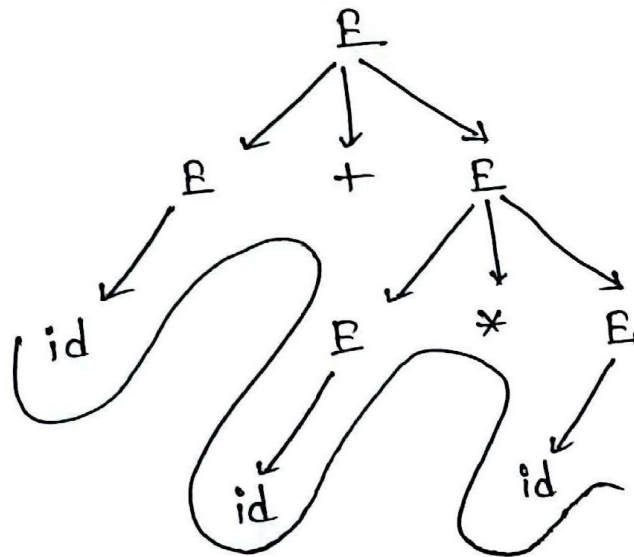
$$E \rightarrow E + E \mid E * E \mid E + E * E$$

$$E \rightarrow E + E * id \mid id + id * id$$

$$E \rightarrow E + id * id \mid id + id * id$$

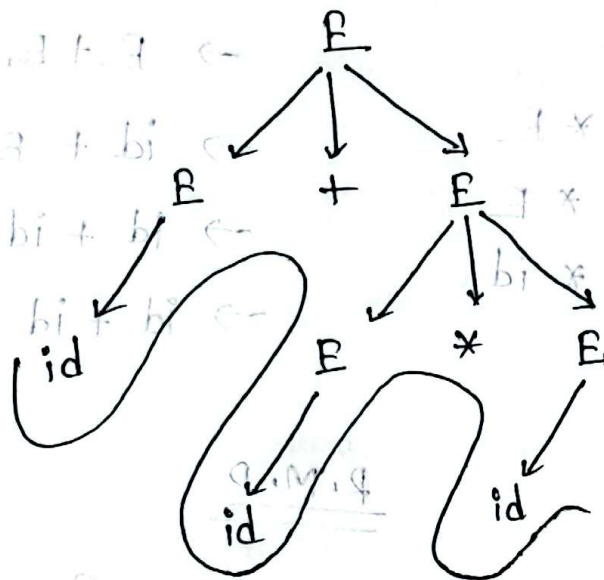
$$E \rightarrow id + id * id$$

Parse tree generation according to the Rightmost Derivations.



string = id + id * id

Parse tree generation according to the Rightmost Derivations.



string = id + id * id

□ Ambiguous Grammar $\rightarrow$

A grammar is said to be ambiguous if there exists more than one leftmost derivations or more than one rightmost derivation or more than one parse tree for the input string.

Given a context free grammar

$E \rightarrow E + E \mid E * E \mid id$ &

string : id + id * id

L.M.D

$$E \rightarrow E + E$$

$$\rightarrow id + E$$

$$\rightarrow id + E * E$$

$$\rightarrow id + id * E$$

$$\rightarrow id + id * id$$

$$E \rightarrow E * E$$

$$\rightarrow E + E * E$$

$$\rightarrow id + E * E$$

$$\rightarrow id + id * E$$

$$\rightarrow id + id * id$$

R.M.D

$$E \rightarrow E + E$$

$$\rightarrow E + E * E$$

$$\rightarrow E + E * id$$

$$\rightarrow E + id * id$$

$$\rightarrow id + id * id$$

$$E \rightarrow E * E$$

$$\rightarrow E * id$$

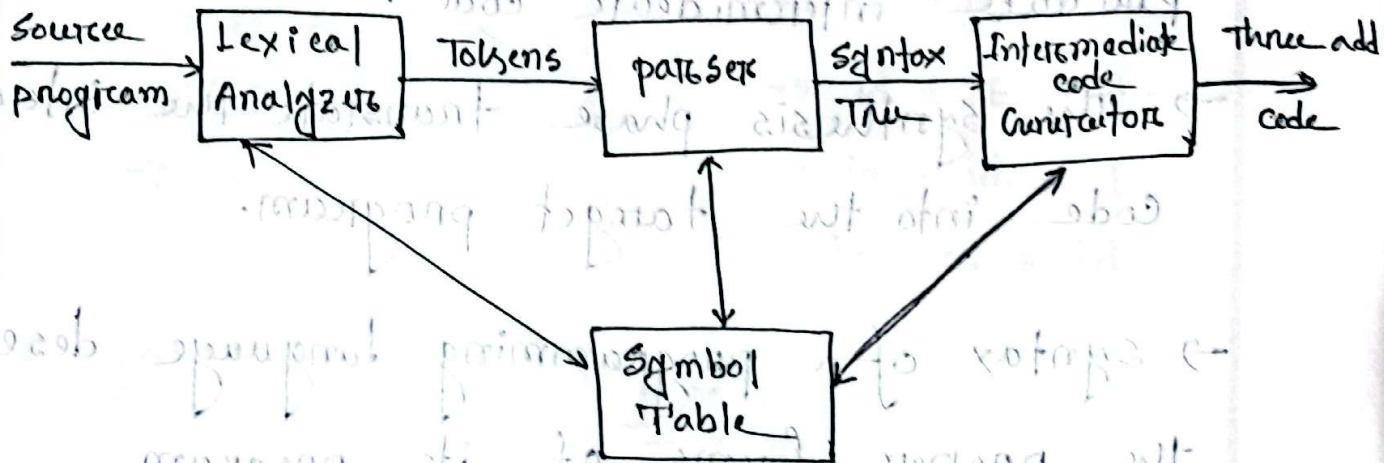
$$\rightarrow E + E * id$$

$$\rightarrow E + id * id$$

$$\rightarrow id + id * id$$

- In analysis phase of a compiler breaks up a source program into constituent pieces and produce intermediate code.
- The synthesis phase translate the intermediate code into the target program.
- Syntax of a programming language describes the proper forms of its program
- Semantics of the language defines what its program mean, that is, what each program does when it executes.
- For specifying syntax, we present a widely used notations called context free grammars or BNF.
- Beside specifying the syntax of a language a context free grammar is used to help guide the translation of programs.

Q. A model of a compiler front end.



→ A lexical analyzer allows the translator to handle multicharacter constructs like identifiers, which are written as sequence of characters, but are treated as units called tokens during syntax analysis.

Ex: Count + 1

Here "count" is treated as a single unit.

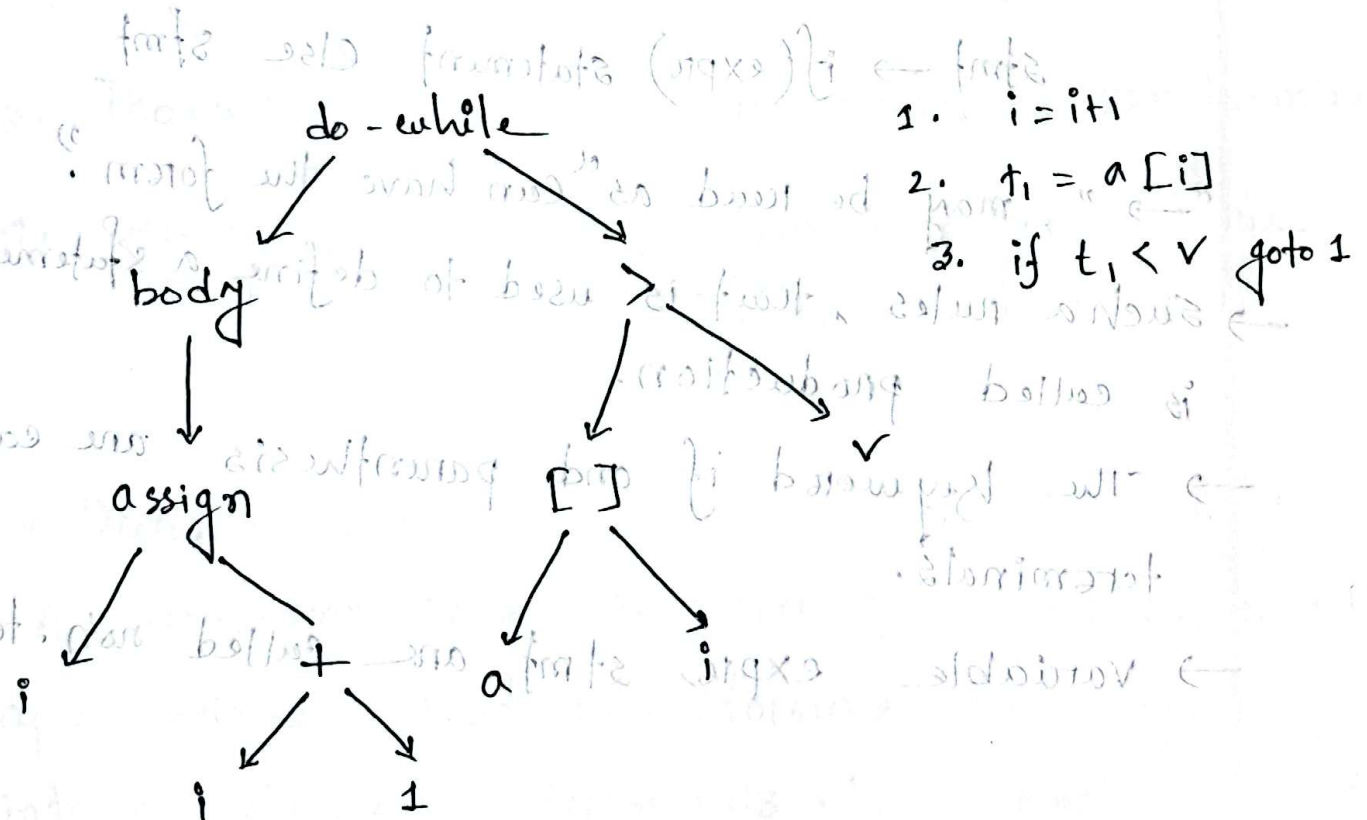
Sub.:

SAT ☐ SUN ☐ MON ☐ TUE ☐ WED ☐ THU ☐ FRI ☐

DATE: / /

Next we consider intermediate ^{code} generation. Two forms of intermediate code, one is "abstract syntax tree" or "simply syntax tree", represents the hierarchical syntactic structure of the source program.

The other common intermediate representation is a sequence of "three-address" instructions.



Intermediate code for

"do $i = i + 1$; while ($a[i] < v$);"

▣ Syntax Definition →

"context free grammars" or "grammars" that is used to specify the syntax of a language.

A grammar naturally describes the hierarchical structure of most programming language constructs.

$stmt \rightarrow if(expr) \text{ statement } else \text{ stmt}$

" $\rightarrow$ ", may be read as "can have the form".

→ Such a rule, that is used to define a statement is called production.

→ The keyword `if` and parenthesis are called terminals.

→ variable `expr`, `stmt`, are called non-terminals.

Sub.:

SAT ☐ SUN ☐ MON ☐ TUE ☐ WED ☐ THU ☐ FRI ☐

DATE: / /

B Definition of Grammars $\rightarrow$

A context free grammar has four components

1. Terminal $\rightarrow$ sometime referred to as tokens. Are ($\leq$, $<$, boldface the elementary symbols of the language string) - are defined by the grammar.
2. Non-terminals $\rightarrow$ sometime referred to as (non-italicized name or symbol) syntactic variables.
3. Productions $\rightarrow$ Production consists of non-terminals
4. Start symbol $\rightarrow$ A designation of one of the non-terminals.

B Derivations $\rightarrow$

A derivation is a sequence of grammar rule applications that transform a start symbol into a string of terminals. It demonstrates how a particular string in the language of the grammar can be generated or derived step by step.

Example 2.1: $\rightarrow$ Considering a context free grammar string are $9-5+2$. Since a plus or minus sign must appear between two digits. The following grammar describes the syntax of these expression. The productions are

$$\text{list} \rightarrow \text{list} + \text{digit}$$

$$\text{list} \rightarrow \text{list} - \text{digit}$$

$$\text{list} \rightarrow \text{digit}$$

$$\text{digit} \rightarrow 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$$

The bodies of the three productions with non-terminal list as head equivalently can be grouped.

$$\text{list} \rightarrow \text{list} + \text{digit} \mid \text{list} - \text{digit} \mid \text{digit}$$

According to our conventions, the terminals of the grammar are the symbols

Sub.:

SAT ☐ SUN ☐ MON ☐ TUE ☐ WED ☐ THU ☐ FRI ☐

DATE: / /

+ - 0 1 2 3 4 5 6 7 8 9

The string of zero terminals, written as ϵ , is called empty string.

* Terminals $\rightarrow$

Terminals are the basic symbols from which strings in the language are formed. Terminals can not be broken down any further - they are the atomic units of the grammar. Terminal could be -

- + (plus)
- - (minus)
- * (multiplication)
- / (Division sign)
- 1, 2, 3, ..., n etc (numeric digits)

* Non-terminals $\rightarrow$

Non terminals are symbol that represent syntactic categories or groups of symbols in the language.

Non terminals could be -

- Expression (for an arithmetic exp)
- Term (for terms in the exp)
- factor (for individual factors in a term)

* Production $\rightarrow$

Expression $\rightarrow$ Expression + Term

Expression $\rightarrow$ Term

Term $\rightarrow$ Term * factor

Term $\rightarrow$ factor

factor $\rightarrow$ (Expression)

factor $\rightarrow$ numbers.

* Start symbol $\rightarrow$

The start symbol is special non-terminal that represents the starting point of the grammar.

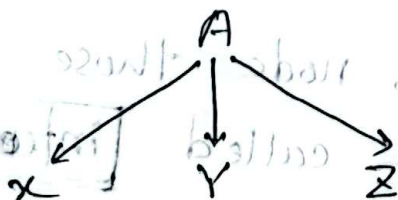
In. Here, Expression is start symbol.

Expression $\rightarrow$ Expression + Term

* Parsing have problem of taking a string of terminals and figuring out how to derive it from the start symbol of the grammar, and if it can't be derived from the start symbol of the grammar, then reporting syntax errors within the string.

Parse Tree $\rightarrow$

A parse tree pictorially shows how the start symbol of a grammar derives a string in the language. If nonterminal A has a production $A \rightarrow XYZ$.



This parse tree have a following properties

- $\rightarrow$ Root is labeled by the start symbol.
- $\rightarrow$ Each leaf is labeled by a terminal or by ϵ .
- $\rightarrow$ Every interior node is labeled by a nonterminal.

Tree Terminology →

→ A tree consists of one or more nodes.

Nodes may have labels.

→ Exactly one node is the root. All nodes except the root have unique parent, the root has no parent.

→ If node N is the parent of node M , then M is the child of N . The children of one node are called siblings.

→ A node with no children is called a leaf. Other nodes, those with one or more children are called interior node.

Example: 2.4

The derivation of $9-5+2$ string. This string are following by the given grammar of production set.

Sub.: _____

SAT ☐ SUN ☐ MON ☐ TUE ☐ WED ☐ THU ☐ FRI ☐

DATE: / /

$list \rightarrow list + digit / list - digit / digit$

$digit \rightarrow 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$

Solution:

$list \rightarrow list + digit$
 ~~$\rightarrow digit +$~~

$list \rightarrow list - digit + digit$

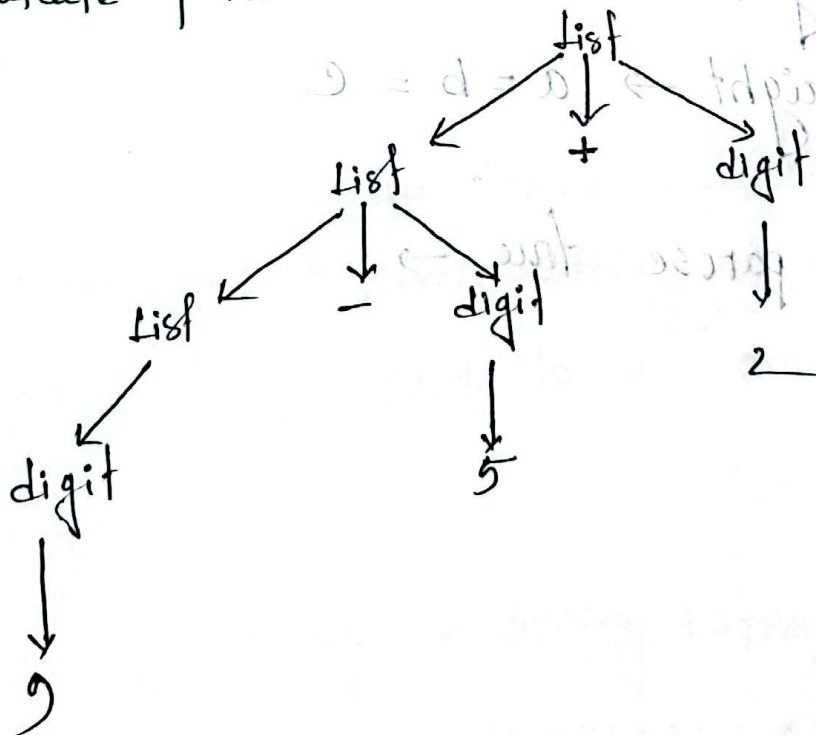
$list \rightarrow digit - digit + digit$

$list \rightarrow 9 - digit + digit$

$list \rightarrow 9 - 5 + digit$

$list \rightarrow 9 - 5 + 2$

generate parse tree



Ⓛ Associativity of operators $\rightarrow$

right $\rightarrow$ letters = right || letters

letters $\rightarrow a|b|\dots|z$

string, $a = b = c$

right $\rightarrow$ letters = right

right $\rightarrow a =$ right

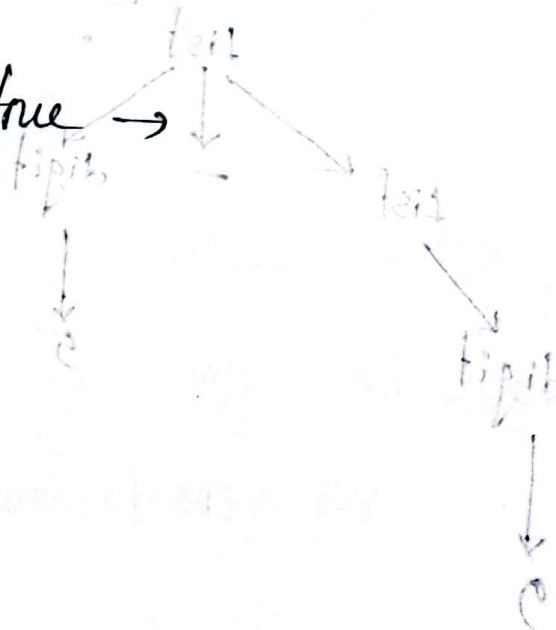
right $\rightarrow a =$ letters = right

right $\rightarrow a = b =$ right

right $\rightarrow a = b =$ letters

right $\rightarrow a = b = c$

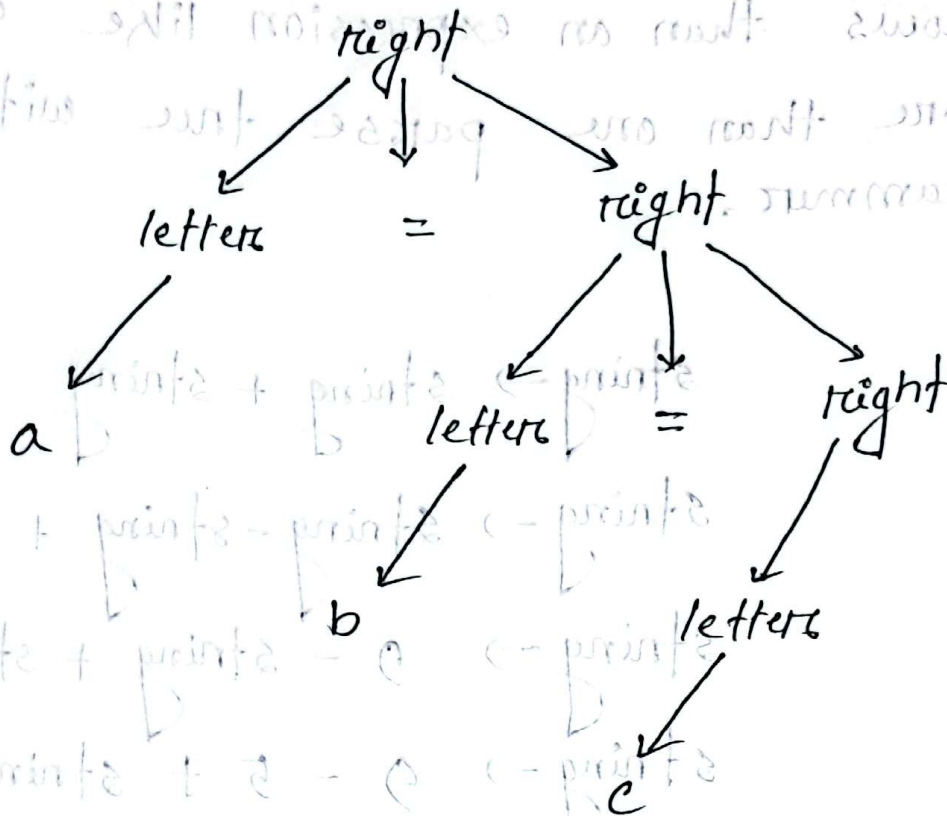
construct parse tree $\rightarrow$



Sub: _____

SAT	SUN	MON	TUE	WED	THU	FRI
☐	☐	☐	☐	☐	☐	☐

DATE: / /



parse tree for left and right associative grammars.

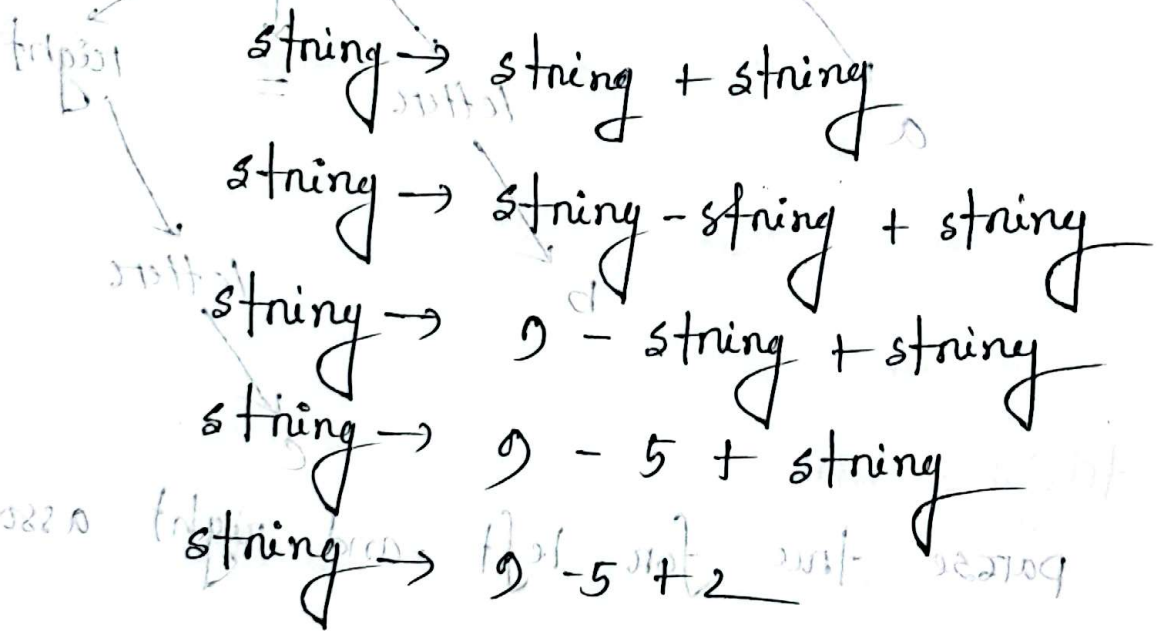
ⓑ Ambiguity →

A grammar can have more than one parse tree generating a given string of terminals. Such a grammar is said to be ambiguous.

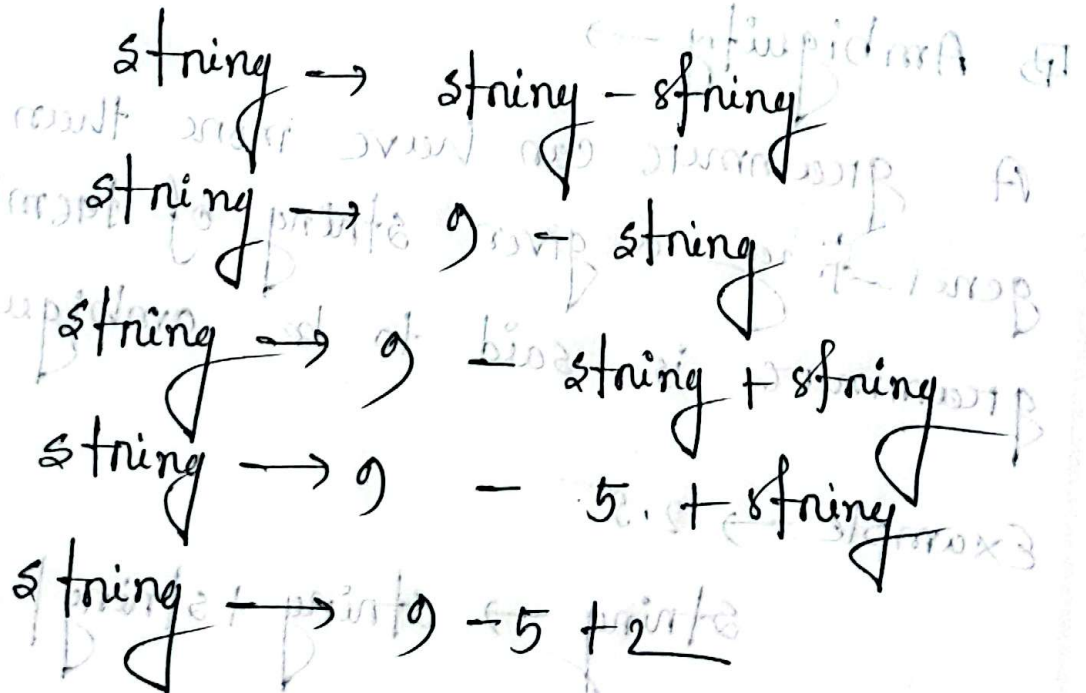
Example → 2.5

string → string + string | string - string | 0 | 1
 12 | 3 | 4 | 5 | 6 | 7 | 8 | 9 .

Shows that an expression like $9-5+2$ has more than one parse tree with this grammar.



another

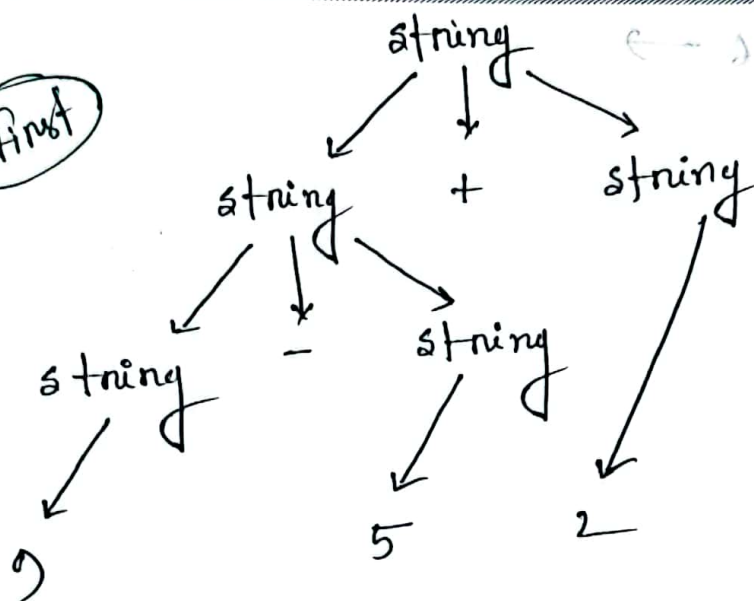


Sub.:

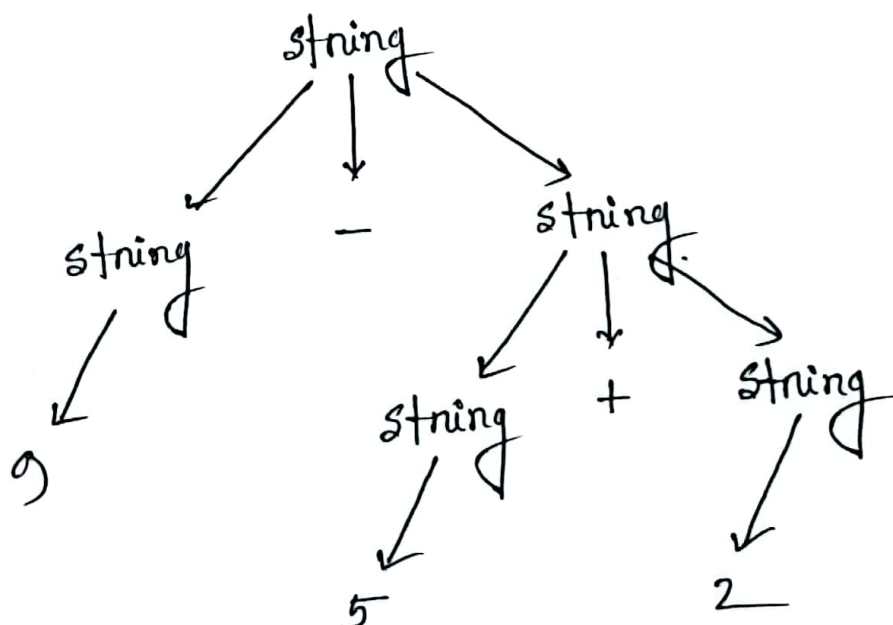
SAT ☐ SUN ☐ MON ☐ TUE ☐ WED ☐ THU ☐ FRI ☐

DATE: / /

(First)



(Second)



we see that the two parse tree are constructed for one expression, it means that it's has ambiguity.